MISSION DE LA SÉANCE

Google ADK en action. Un agent sous contrôle.

Deux tools, une exécution observable et un serveur MCP local.
Le runtime utilisé en S2 devient explicite.

Pas de nouvelle architecture appliquée

→ pour commencer

Photo Florian Olivo / Unsplash

LA CIBLE

L’agent choisit une action. Votre code garde la décision finale.

TOOLS

Deux capacités

Entrées validées, sorties bornées, erreurs explicites.

BOUCLE

Cinq tools max.

Chaque appel apparaît dans une trace observable.

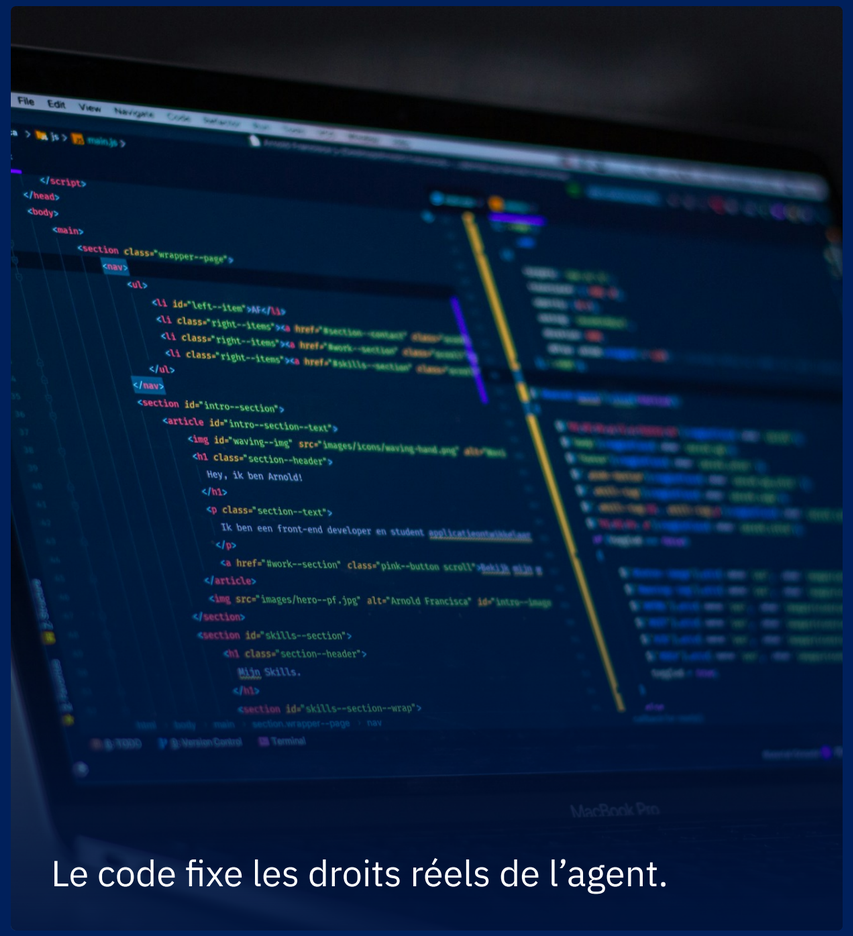
MCP

Lecture seule

Aucun SQL, shell, secret ou écriture arbitraire.

SE REPÉRER AVANT DE CODER

Observer la démo, puis contrôler ses décisions.



Le code fixe les droits réels de l’agent.

FOURNI

Une démo ADK locale et deux tools.

Ouvrir `tp/s3-agents-mcp/adk_demo/` : agent, runner et serveur MCP.

À ADAPTER

Vos capacités métier, typées.

Partir de `search_library` et de l’historique ; garder des entrées bornées et des sorties explicites.

À CRÉER

Les garde-fous et leurs tests.

Faire respecter cinq tools, six appels modèle et trente secondes. Rendre l’arrêt explicable.

Photo Florian Olivo / Unsplash · Pictogrammes : tracés Lucide, licence ISC.

PRÉPARER L'ENVIRONNEMENT

Retrouvez ADK et branchez vos capacités métier.

1

Rejouer les tests S2

Le chat fake et le RAG doivent encore fonctionner.

2

Choisir deux actions

Exemples : rechercher, calculer, filtrer, lire un historique.

3

Écrire les limites

Ce que chaque tool peut faire… et surtout ne pas faire.

Installation cumulative

S3 utilise encore ce qui a été construit en S2.

```
uv sync --extra rag \
  --extra agent --group dev
```

CONCEPT 1 · AGENT

L’Agent décrit ce que l’assistant peut faire.

```
root_agent = Agent(
    name="music_discovery",
    model=build_adk_model(),
    instruction="Réponds avec les outils
autorisés.",
    tools=[get_listening_history,
           catalogue],
)
```

Instructions
Le rôle et les règles exprimées au modèle.

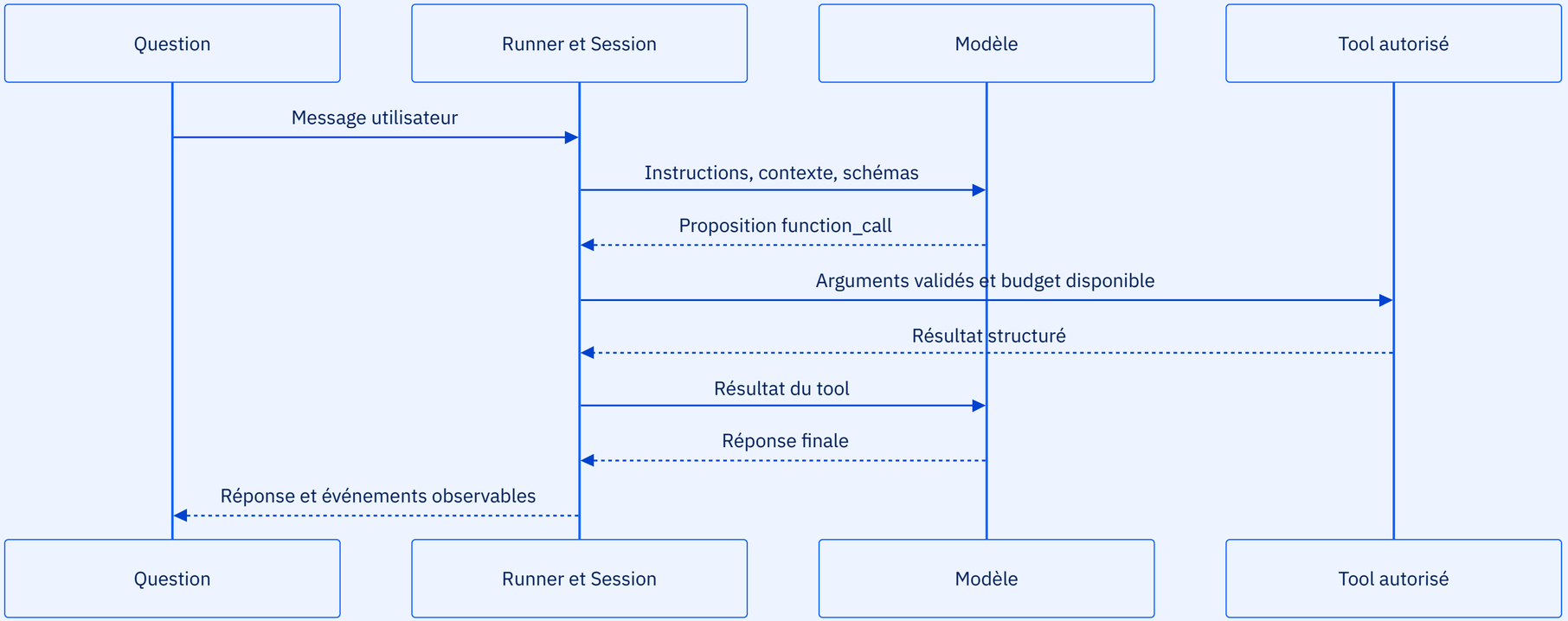
Modèle
Cohere, Gemini ou une simulation explicite.

Tools
Les seules capacités exposées à l’agent.

Agent ≠ modèle. Le modèle propose un appel ; l’application contrôle son exécution.

CONCEPT 2 · RUNNER

Le Runner fait circuler messages et résultats.



Flux d’exécution simplifié. Les événements rendent visibles les échanges ; ils ne dévoilent pas le raisonnement caché du modèle.

CONCEPT 3 · SESSION ET STATE

Le contexte d’exécution a une durée de vie.

Une session ADK

```
sessions = runner.session_service
session = await sessions.create_session(
        app_name="s3_discovery",
        user_id="local",
    )
```

Messages et état d’une exécution, isolés par session.

Starter / CLI

Une session mémoire éphémère par question.

ADK Web

Des sessions de développement conservées localement.

Données métier

Catalogue, documents et règles restent dans l’application.

L’historique affiché par Streamlit n’est pas automatiquement une mémoire transmise au modèle.

CONCEPT 4 · EVENT

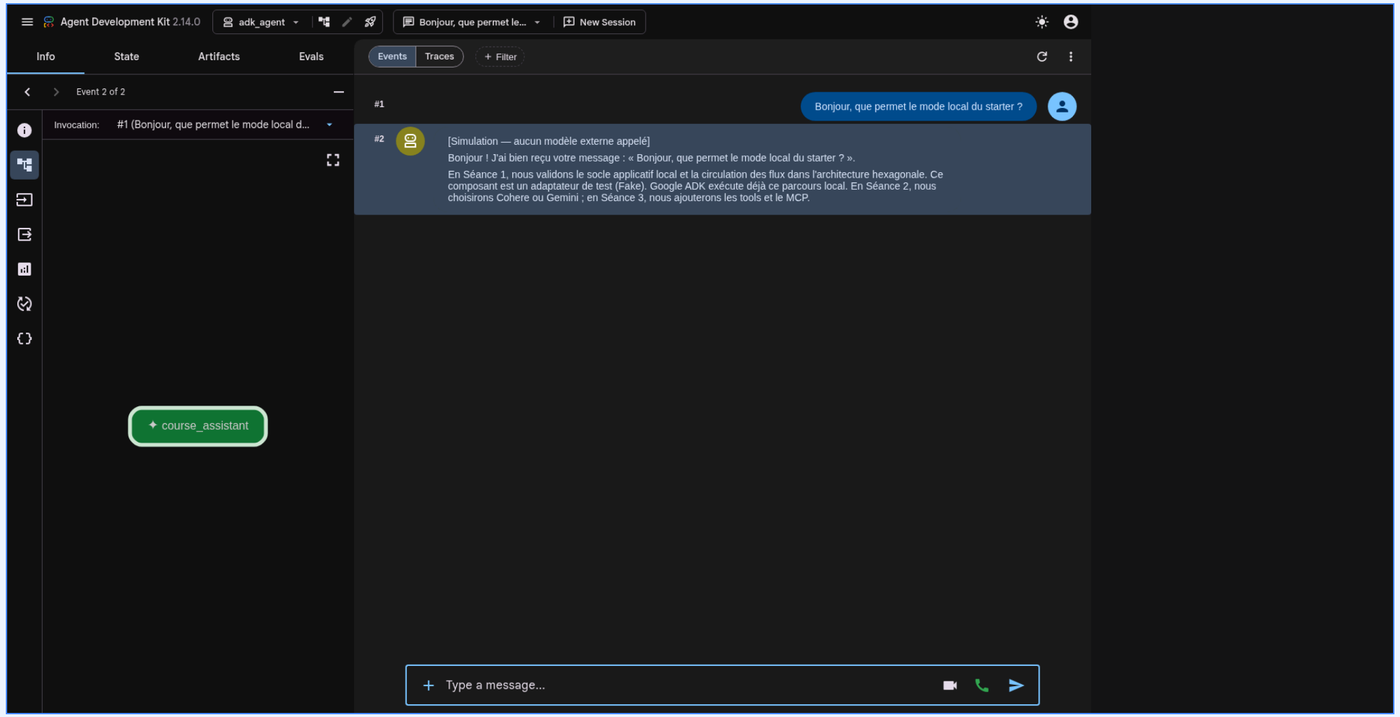
Une trace raconte les échanges exécutés.

01	Message utilisateur	« Quels morceaux focus ? »
02	function_call	search_library(query="focus")
03	function_response	items + count, validés avant réutilisation
04	Réponse finale	Réponse fondée sur les résultats disponibles

Un appel proposé n’est pas encore un tool exécuté. Une trace permet de vérifier nom, arguments, résultat et nombre d’appels.

OBSERVER AVANT DE MODIFIER

ADK Web permet d’inspecter un échange.



Chat
Envoyer la question de test.

Events / Traces
Repérer appels, résultats et réponse finale.

State
Identifier le contexte de la session.

```
uv run adk web tp/s3-agents-mcp/adk_demo --port 8001
```

CONTRAT AVANT CODE

FunctionTool expose une fonction typée et validée.

NOM

Verbe + objet

`search_catalog`, pas `do_everything`.

INPUT

Schéma strict

Types, champs obligatoires, bornes et valeurs autorisées.

OUTPUT

Petit résultat

Données utiles, nombre de résultats et erreur actionnable.

- ✓ Refuser les arguments inconnus.
- ✓ Limiter le volume retourné.
- ✓ Tester les entrées invalides.
- ✓ Ne jamais exposer un SDK brut.

ATELIER · FUNCTIONTOOL

Partez d’un tool pur avant de le confier à l’agent.

01 **Rejouer la démo fournie**

Lancer le script local et lire sa trace. Le profil fake montre un plan déterministe, pas une décision intelligente.

02 **Typer une capacité de votre domaine**

Arguments courts, validation dans la fonction et résultat sérialisable. Tester la fonction directement.

03 **Exposer et observer**

Envelopper avec FunctionTool ; ajouter à la liste autorisée. Dans ADK Web, relier appel, résultat et réponse finale.

FOURNI · lancer depuis la racine du starter

```
uv sync --extra agent --group dev
uv run python tp/s3-agents-mcp/adk_demo/run.py

uv run adk web tp/s3-agents-mcp/adk_demo \
  --host 127.0.0.1 --port 8001
```

Vérifier. La trace contient le nom du tool et ses arguments. Si aucun tool ne part en live, vérifier le schéma exposé et la question posée.

Étapes détaillées et commandes copiables dans le README du TP.

ORCHESTRATION BORNÉE

Décider → valider → exécuter → tracer → recommencer.



- ✓ Au plus cinq appels de tools.
- ✓ Arrêt immédiat après une erreur.
- ✓ Tool inconnu = échec explicite.
- ✓ Chaque étape conserve entrée et sortie.

TROIS ÉTATS OBLIGATOIRES

Une terminaison doit toujours être compréhensible.

COMPLETED

Réponse obtenue

Le Runner reçoit une réponse finale après zéro ou plusieurs tools.

LIMITED

Limite atteinte

Un budget de tools, d’appels modèle ou de temps est atteint.

FAILED

Erreur maîtrisée

Tool inconnu, argument invalide ou exécution impossible.

```
uv run pytest -k agent
```

CONTRÔLER L’EXÉCUTION

Trois budgets distincts protègent le parcours.

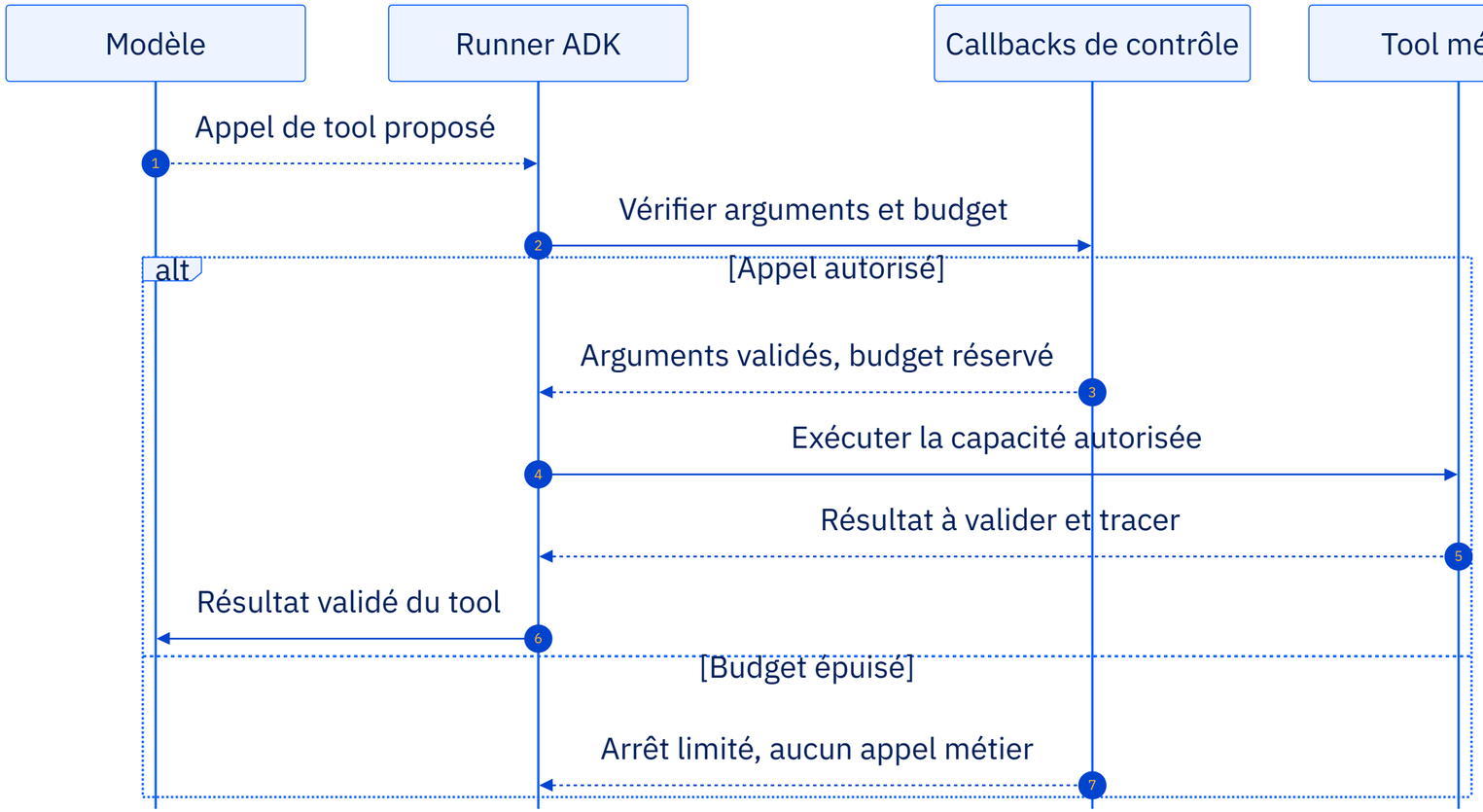
Budget	Contrôle	Ce qu’il borne
`5 tools`	Callback avant chaque appel	Les outils réellement exécutés, même en parallèle
`6 appels modèle`	RunConfig(max_llm_calls=6)	Les décisions successives du modèle
`30 secondes`	asyncio.timeout(30)	La durée totale de l’exécution

Un appel modèle peut proposer plusieurs tools. Une consigne « cinq étapes maximum » ne suffit pas : la limite s’applique avant l’effet.

completed : réponse finale · limited : budget atteint · failed : erreur explicite.

FLUX À IMPLÉMENTER · SÉQUENCE

Une proposition du modèle ne vaut pas autorisation.



Les callbacks refusent les arguments invalides et réservent le budget avant l’effet métier. Flèche pleine : appel ; pointillés : retour.

ATELIER · LIMITES ET MCP

Épuisez le budget avec un scénario contrôlé.

01 **Créer un modèle de test qui insiste**

Faire proposer plus de cinq tools. Compter les appels effectivement exécutés, pas seulement les événements reçus.

02 **Séparer les trois limites**

Compteur de tools, maximum de six appels au modèle et délai global de trente secondes : chacun a son test.

03 **Tester aussi la frontière MCP**

Arrêter le serveur ou retourner une sortie mal formée. Valider la forme reçue et produire une erreur publique traçable.

À CRÉER · tests/test_agent_limits.py · scénario

```
# Modèle fake : propose 6 appels de tool.
result = run_agent.execute(question)

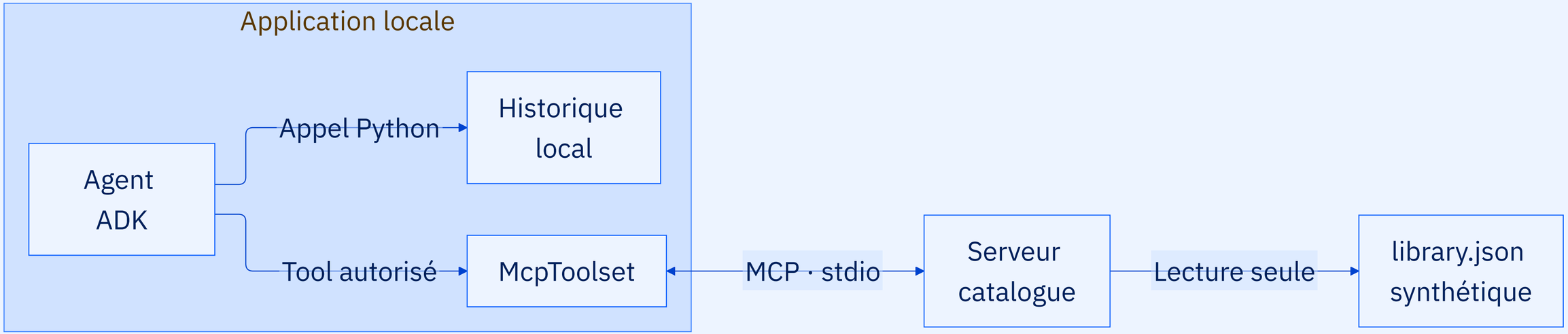
assert result.status == "limited"
assert executed_tools <= 5

# Autres tests : arguments invalides,
# délai expiré et serveur MCP absent.
```

Vérifier. Un arrêt limited ou failed est visible, avec les étapes déjà réalisées. Aucun sixième effet métier ne se produit.

MCPTOOLSET · DÉCOUVERTE ET APPEL

Un protocole relie des capacités autorisées.



McpToolset découvre les outils du serveur. tool_filter=["search_library"] expose uniquement le catalogue en lecture seule. Le serveur reçoit le chemin du jeu synthétique, aucune clé LLM.

FRONTIÈRE NON FIABLE

MCP transporte un appel. Il ne donne pas tous les droits.

1

Lancer le serveur local

Un processus stdio, une fixture synthétique, zéro secret.

2

Autoriser un nom précis

Le client possède une allowlist de tools MCP.

3

Valider la sortie

Traiter toute réponse MCP comme une entrée externe.

Interdictions

- pas de requête SQL libre ;
- pas de commande shell ;
- pas d’écriture de fichier ;
- pas de secret dans les arguments.

SCÉNARIO DE VALIDATION

Montrez le chemin heureux puis deux garde-fous.

01

Succès

Question → tool autorisé → réponse finale + trace.

02

Limite

Modèle simulé qui propose un sixième appel de tool.

03

Refus

Tool inconnu ou argument hors contrat.

```
uv run pytest  &&  uv run ruff check .
git add .  &&  git commit -m "Ajoute un agent borné et le MCP local"
```

AVANT DE QUITTER S3

L’agent agit dans une boîte clairement délimitée.

Vous avez construit

- deux tools métier validés ;
- un Runner ADK et cinq tools maximum ;
- trois états de terminaison ;
- un appel MCP en lecture seule.

À conserver pour S4

- Agent, Runner, Session et Event ;
- la trace des exécutions ;
- les allowlists ;
- les erreurs explicites.

```
uv run pytest  &&  uv run ruff check .
```

S3 · CONSIGNES À GARDER PROJETÉES

À vous : un agent observable et borné.

01 **Rejouer et lire la démo**

Repérer Agent, Runner, Session, événements et tools dans le dossier adk_demo.

02 **Adapter deux tools métier**

Typer et tester les fonctions avant FunctionTool ; garder un résultat petit et sérialisable.

03 **Relier un tool par MCP**

Filtrer les noms autorisés, valider entrées et sorties ; tester le serveur indisponible.

04 **Forcer les limites**

Cinq tools, six appels modèle, trente secondes ; tracer l'arrêt et ses causes.

À montrer avant de terminer

- Un parcours utile avec appels et résultats.
- Une entrée refusée avant exécution.
- Un budget atteint et un échec MCP expliqués.

```
uv run python \
  tp/s3-agents-mcp/adk_demo/run.py
uv run pytest
```

Bloqué ? README de ce TP → « Pas à pas ». Commencer en fake ; expliquer les états completed, limited et failed.